

CS 4373/6373 — CUDA Hello World Lab Report

Author: Mikey Ferguson **Course:** CS 4373/6373 — High Performance Computing **Lab:** CUDA Hello World on Nautilus **Date:** 2026-05-08 **Repository:** [CS-4373-01-SP26/cuda-hello-world-lab-nautilus-orientation-Pirate-Hunter-Zoro](#)

Submission Status

Item	Status
Code pushed to GitHub Classroom repo	Complete
.github/workflows/build-check.yml (compile-only CI on ubuntu-22.04 , sm_70)	Passing
cuda_hello.cu compiled with nvcc on Nautilus pod	Complete
Binary executed on real GPU hardware in tandy-hpc namespace	Complete
Output captured in run_10.txt and gpu_info.txt	Complete
Pod deleted after session	Complete

The most recent commit on [main](#) is [c7b34a5](#) — [Ran CUDA hello world on Nautilus](#), which triggered the GitHub Actions build check. The workflow compiled [cuda_hello.cu](#) cleanly on a GPU-less Ubuntu runner with [nvcc -arch=sm_70 -O2](#), confirming the source builds correctly.

Environment

- **Cluster:** Nautilus (NRP), namespace [tandy-hpc](#)
- **Pod manifest:** [k8s/gpu-dev-pod.yaml](#)
- **Container image:** [nvidia/cuda:12.2.0-devel-ubuntu22.04](#)
- **Driver / CUDA on node:** [595.71.05](#) / CUDA [13.2](#) (per [nvidia-smi](#))
- **Build command:** [make](#) (auto-detected [sm_75](#) from [nvidia-smi](#))

Captured GPU snapshot ([gpu_info.txt](#), head):

```
NVIDIA-SMI 595.71.05    Driver Version: 595.71.05    CUDA Version: 13.2
GPU   Name                Memory              Util
 0   NVIDIA GeForce RTX 2080 Ti    11264 MiB          100%
...
```

Reflection

Q1 — GPU model and specs

The GPU allocated to my pod was an **NVIDIA GeForce RTX 2080 Ti** (Turing architecture, TU102).

Property	Value
Compute capability	7.5 (sm_75)
Streaming multiprocessors (SMs)	68
CUDA cores	4 352 (64 FP32 cores per SM × 68 SMs)
Memory	11 GB GDDR6
Max threads per block	1 024
Max threads per SM	1 024

The Makefile's auto-detection (`nvidia-smi --query-gpu=compute_cap`) returned 7.5, so the build line was `nvcc -arch=sm_75 -O2 -o cuda_hello cuda_hello.cu`.

Q2 — Output ordering across repeated runs

Running `./cuda_hello 10` five times in a row produced **identical, in-order output** on every run (full transcript in `run_10.txt`):

```

--- run 1 ---
Hello from thread 0!
Hello from thread 1!
Hello from thread 2!
Hello from thread 3!
Hello from thread 4!
Hello from thread 5!
Hello from thread 6!
Hello from thread 7!
Hello from thread 8!
Hello from thread 9!
--- run 2 ---      (identical)
--- run 3 ---      (identical)
--- run 4 ---      (identical)
--- run 5 ---      (identical)

```

With `<<<1, 10>>>`, all ten threads belong to a single block and a single warp (a warp is 32 threads on every NVIDIA GPU through Turing). Threads in one warp have their outputs emitted in order (0, 1, 2, ..., 31) after `cudaDeviceSynchronize()`

Thread output order is **not guaranteed** across multiple warps or multiple blocks, warps are scheduled independently, and the buffer flush interleaves their writes. Order is only stable here because the entire kernel fits inside one warp, which runs has threads run sequentially.

Q3 — Meaning of the 1 in `<<<1, thread_count>>>`

In a CUDA launch configuration `<<<gridDim, blockDim>>>`, the first argument is the **number of thread blocks in the grid**. So `<<<1, thread_count>>>` launches a grid containing one block of `thread_count` threads. Those threads are partitioned into warps of size at most 32.

Q4 — Maximum thread count

The hard ceiling for a single-block launch on compute capability 7.5 is **1 024 threads per block** (Pacheco §6.7, Table 6.3, `maxThreadsPerBlock`). I confirmed this empirically on the pod:

- `./cuda_hello 1024` — the kernel launched and ran. Output contained "Hello from thread *k*!" lines for `k = 0 ... 1023`
- `./cuda_hello 1025` — the kernel **failed to launch**. Effectively no greetings were produced (only 0 lines of kernel output), because `blockDim.x = 1025` exceeds the device's `maxThreadsPerBlock = 1024`. The CUDA runtime rejects the launch with `cudaErrorInvalidConfiguration` and the kernel never executes on any SM.

Q5 — Why `cudaDeviceSynchronize()` is necessary

CUDA kernel launches are **asynchronous** with respect to the host (Pacheco §6.5). The triple-angle-bracket call `Hello<<<1, thread_count>>>()`; only enqueues the kernel onto the default stream and returns immediately to the CPU. Before printing thread results, we have to wait for the threads to actually get their results

Files of Record

File	Purpose
<code>cuda_hello.cu</code>	Pacheco §6.4.1 kernel — unmodified
<code>Makefile</code>	<code>nvcc</code> build with auto-detected <code>-arch=sm_XX</code>
<code>k8s/gpu-dev-pod.yaml</code>	Single-GPU dev pod spec for <code>tandy-hpc</code>
<code>.github/workflows/build-check.yml</code>	Compile-only CI (5 pts, passing)
<code>run_10.txt</code>	Captured output of five <code>./cuda_hello 10</code> runs
<code>gpu_info.txt</code>	<code>nvidia-smi</code> snapshot from the Nautilus pod
<code>REPORT.md</code>	This report

Compiling to PDF

From the repo root:

```
pandoc REPORT.md -o REPORT.pdf \
  --from=gfm \
  --pdf-engine=xelatex \
  -V geometry:margin=1in \
  -V mainfont="Helvetica"
```

(Or any other Markdown→PDF renderer; the document uses only standard GFM features — headings, tables, fenced code blocks.)